

FORMATION · 1 JOURNÉE

Les limites du ML classique, Introduction au Perceptron et réseaux multicouches (MLP)

Du perceptron au réseau
multi-couches : comprendre
comment une machine apprend.

Formateur · **Manitra Ainaharison**

Session matinée · Théorie · ~ 3 h

Le parcours de la matinée

1	Introduction au Deep Learning	20 min
2	Le neurone artificiel	30 min
3	Le perceptron	30 min
4	Les limites du perceptron	20 min
5	Du perceptron au MLP	30 min
6	Comment un réseau apprend	40 min
7	Synthèse et transition vers le TP	10 min

PARTIE 1 · 20 MIN

01

Introduction au Deep Learning

Limites du ML classique · Pourquoi le
DL · Apprendre depuis les données
· Vers les architectures profondes

IA, Machine Learning, Deep Learning

IA

Intelligence Artificielle

Toute technique qui permet à une machine d'imiter une capacité humaine : raisonner, percevoir, décider, dialoguer.

ML

Machine Learning

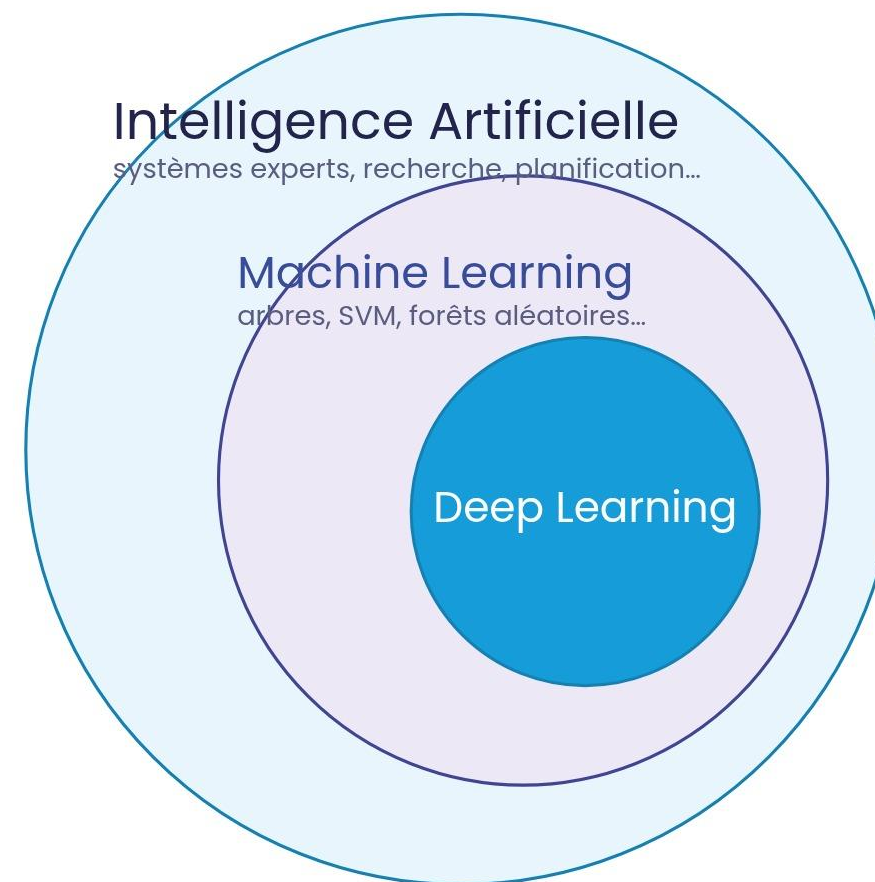
Sous-ensemble de l'IA : au lieu de coder des règles, on montre des exemples et l'algorithme en déduit un modèle.

DL

Deep Learning

Famille du ML où le modèle est un réseau de neurones à plusieurs couches. Extrait automatiquement les caractéristiques.

Trois cercles concentriques



Pourquoi le Deep Learning ?

Les idées existaient depuis les années 1980. Ce qui manquait, ce sont trois ingrédients qui ne se sont réunis que récemment.

1

Données massives

Avant : quelques milliers d'exemples étiquetés à la main. **Aujourd'hui :** Internet, smartphones et capteurs fournissent des millions d'exemples, ce dont les réseaux profonds ont besoin pour bien généraliser.

2

Puissance de calcul

Avant : entraîner un grand réseau prenait des mois sur un processeur classique. **Aujourd'hui :** les GPU (puis les TPU) font le même travail en quelques heures.

3

Nouveaux algorithmes

Avant : on ne savait pas entraîner des réseaux profonds de façon stable. **Aujourd'hui :** backpropagation, activations modernes et régularisation rendent l'entraînement fiable.

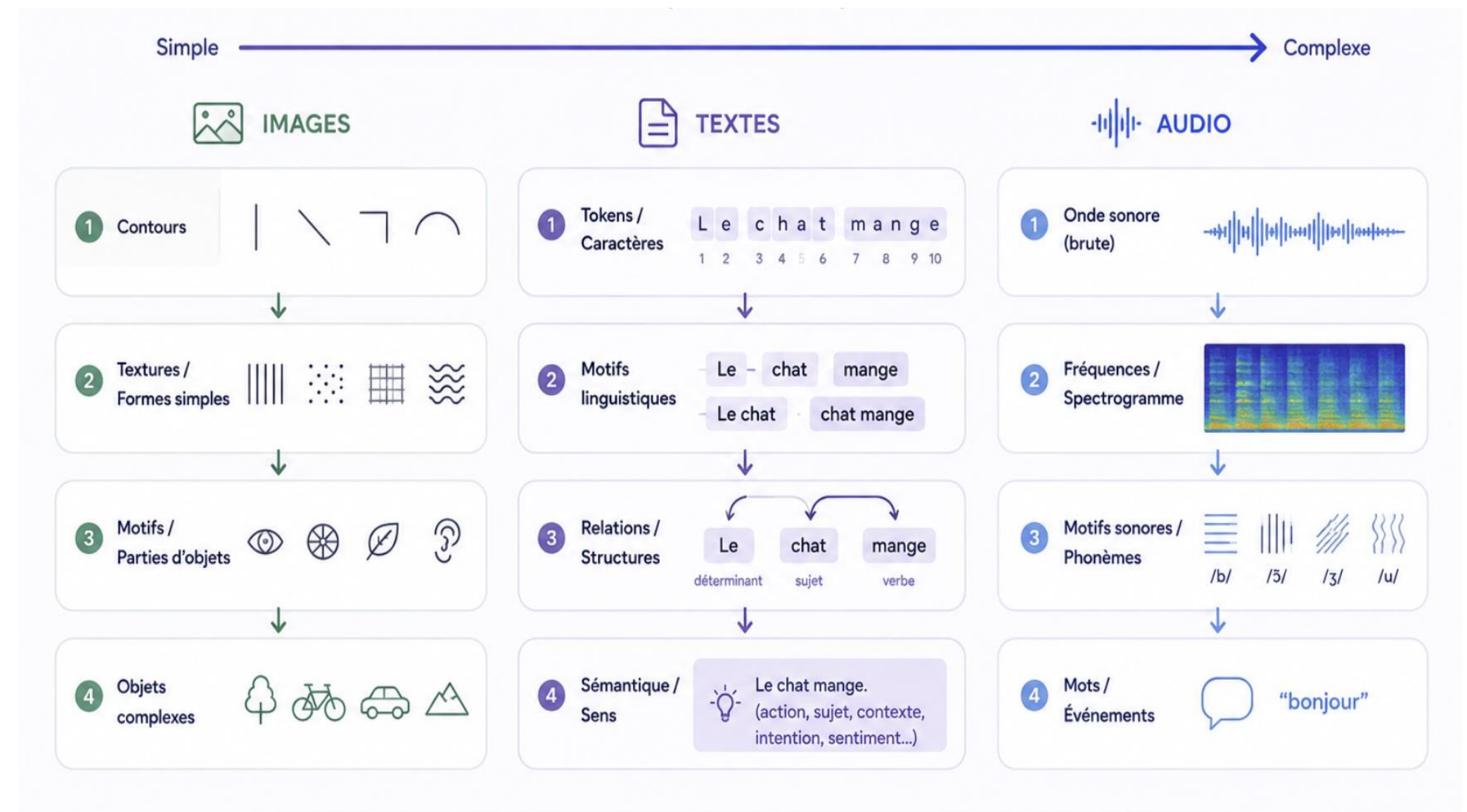
Les limites du ML classique

- **Feature engineering** manuel
- Données **non structurées** difficiles
- Passage à l'échelle limité : **complexité et volume**
- Dépendance à la **qualité des représentations**
- Forte **expertise métier** requise

Pourquoi passer au Deep Learning ?

- Apprentissage **automatique des représentations**
- Features **hiérarchiques** (du simple au complexe)
- Adaptation aux **données complexes**

Images Texte Audio Vidéo
- Exploite de **grandes quantités de données**



Vers des architectures profondes

1

Perceptron

Un seul neurone. Une frontière de décision linéaire.

2

Réseau de neurones

Des neurones en couches. Des frontières combinées, non linéaires.

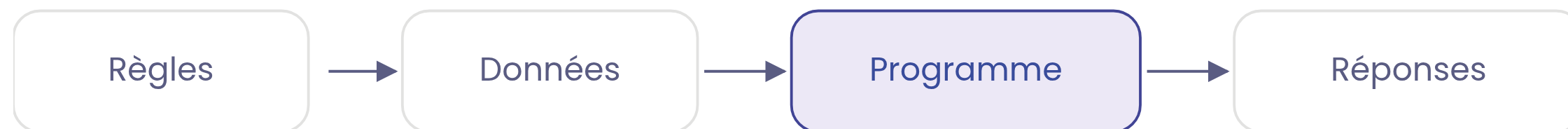
3

Deep Neural Networks

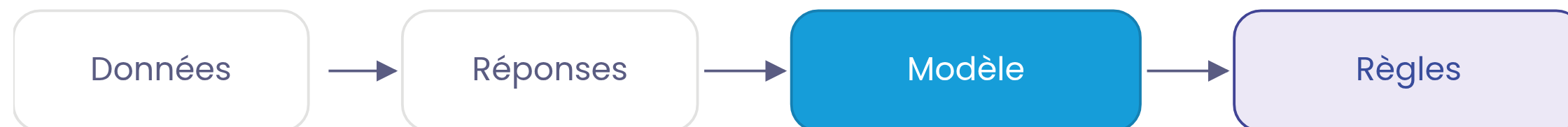
De nombreuses couches profondes. Des représentations riches, apprises seules.

Apprendre à partir des données

Programmation classique



Machine Learning



D'après Chollet : au lieu d'écrire les règles à la main, on les fait émerger d'exemples annotés.

Où trouve-t-on du Deep Learning ?

Vision

Reconnaissance d'objets, imagerie médicale, voitures autonomes, contrôle qualité.

Langage

Traduction, assistants conversationnels, résumé automatique, recherche sémantique.

Audio

Reconnaissance vocale, synthèse, séparation de sources, détection d'anomalies.

Sciences

Repliement des protéines, prévision météo, découverte de matériaux, physique des particules.

PARTIE 2 · 30 MIN

02

Le neurone artificiel

Entrées · Poids · Biais · Somme
pondérée · Activation ·
Intuition géométrique

Du neurone biologique au neurone artificiel

Biologie

Les **dendrites** reçoivent des signaux électriques venant d'autres neurones. Le **corps cellulaire** les intègre. Si l'intégration dépasse un seuil, l'**axone** émet un signal à son tour.

Modèle mathématique

Chaque signal entrant devient un nombre. L'intégration devient une somme pondérée. Le seuil devient une fonction d'activation. Le tout est différentiable, donc apprenable.

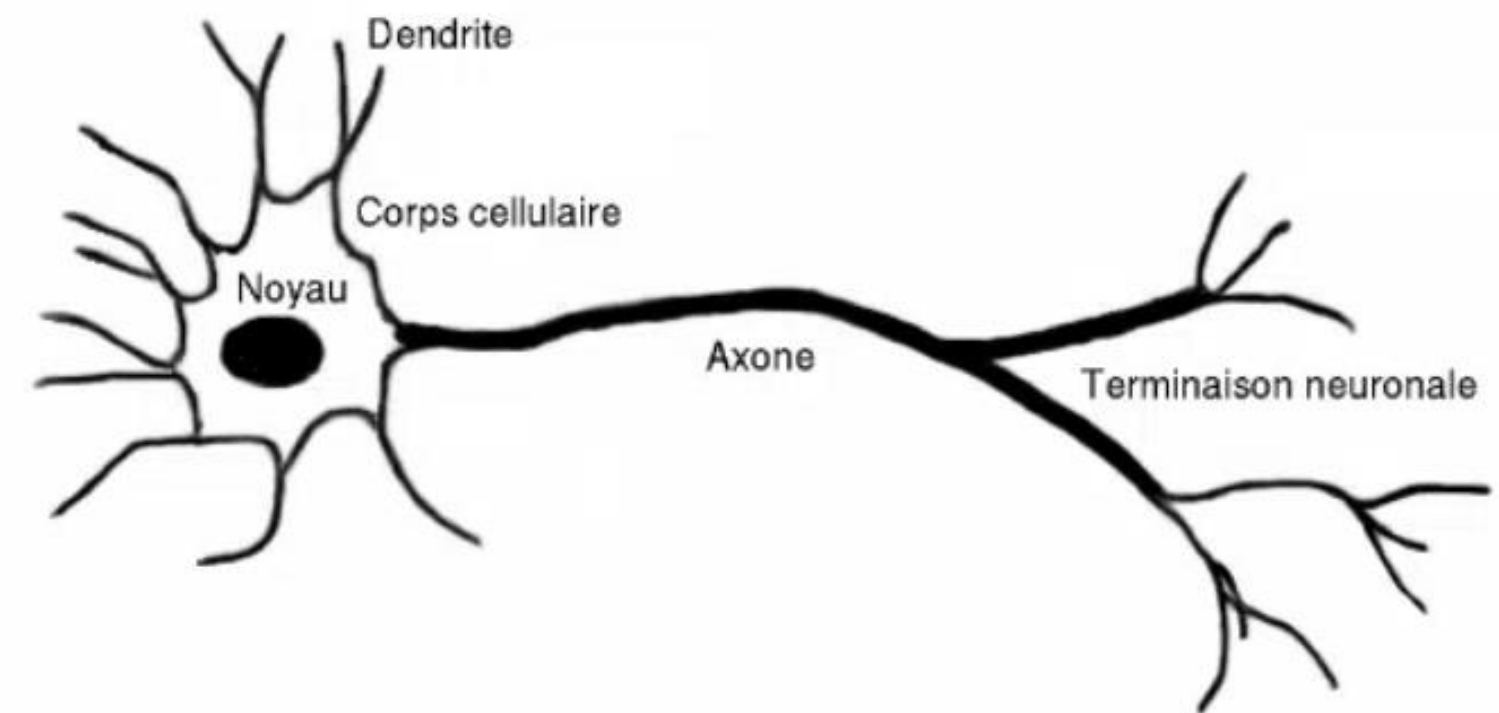
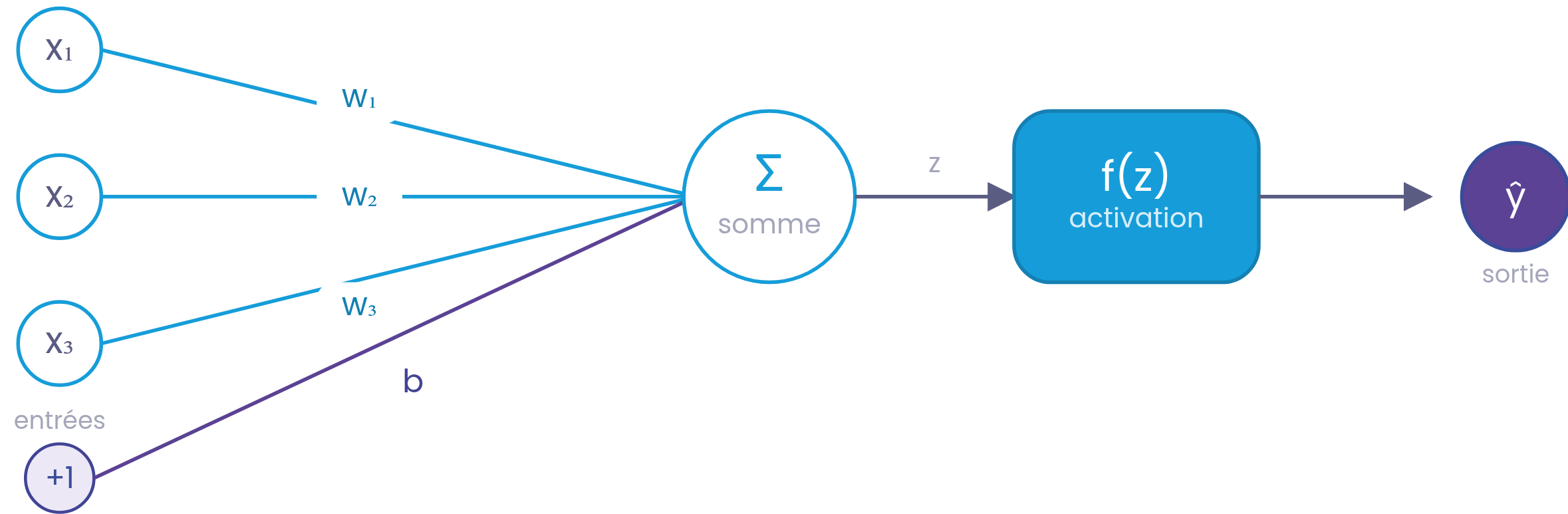


Schéma d'un neurone biologique

Anatomie du neurone artificiel



Entrées : les *features*

Chaque exemple présenté au neurone est un **vecteur de nombres** : ses *features*.

La qualité du modèle dépend directement de la qualité de ces caractéristiques. En deep learning, une bonne partie de ce travail est appris automatiquement par les couches profondes.

EXEMPLES DE FEATURES

Prêt bancaire : âge, revenu, durée, apports

Image : intensité de chaque pixel

Texte : fréquence des mots, embeddings

Capteur : température, pression, vibration

Poids et biais : ce que le neurone apprend

W

Poids

Un nombre par entrée. Positif → l'entrée pousse à activer. Négatif → elle freine. Grand en valeur absolue → grande influence.

b

Biais

Un décalage constant, indépendant des entrées. Sans lui, la frontière de décision passerait toujours par l'origine.

θ

Paramètres θ

Ensemble { poids, biais } : l'objet que l'apprentissage va ajuster. Tout le reste (entrées, fonction d'activation) est fixe.

La somme pondérée

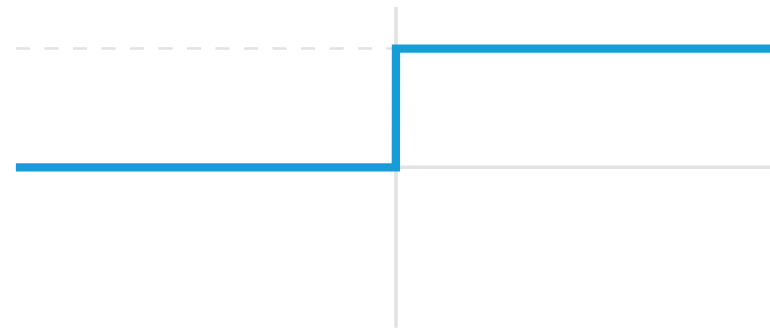
$$Z = W_1 \cdot X_1 + W_2 \cdot X_2 + W_3 \cdot X_3 + b$$

ou, en notation vectorielle : $z = W^T \cdot X + b$

z est le *pré-activation* : un simple nombre réel qui résume l'exemple.

Chaque entrée est **pondérée**, puis toutes les contributions s'ajoutent. Le biais offre un point de départ ajustable. Sur un ordinateur, c'est une seule instruction vectorielle.

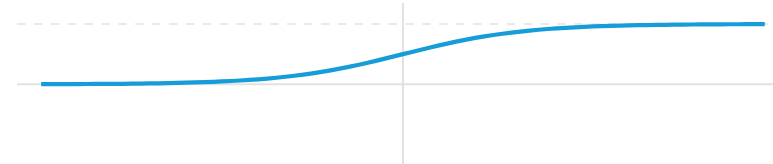
La fonction d'activation



Step (Heaviside)

$$f(z) = 0 \text{ si } z < 0 ; 1 \text{ si } z \geq 0$$

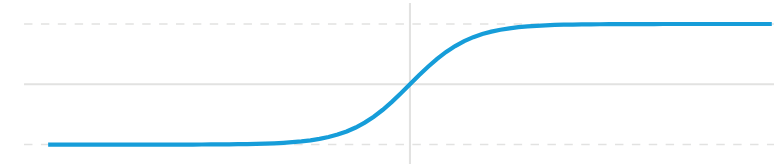
Sortie binaire : le perceptron d'origine.



Sigmoïde

$$\sigma(z) = 1 / (1 + e^{-z})$$

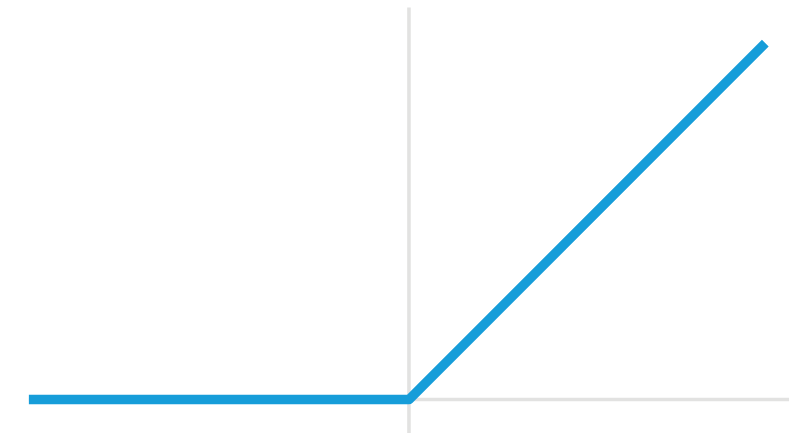
Probabilité entre 0 et 1 :
classification binaire.



Tanh

$$\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$$

Centrée en 0 (-1 à 1) : couches cachées.



ReLU

$$f(z) = \max(0, z)$$

Non saturante : standard des
réseaux profonds.

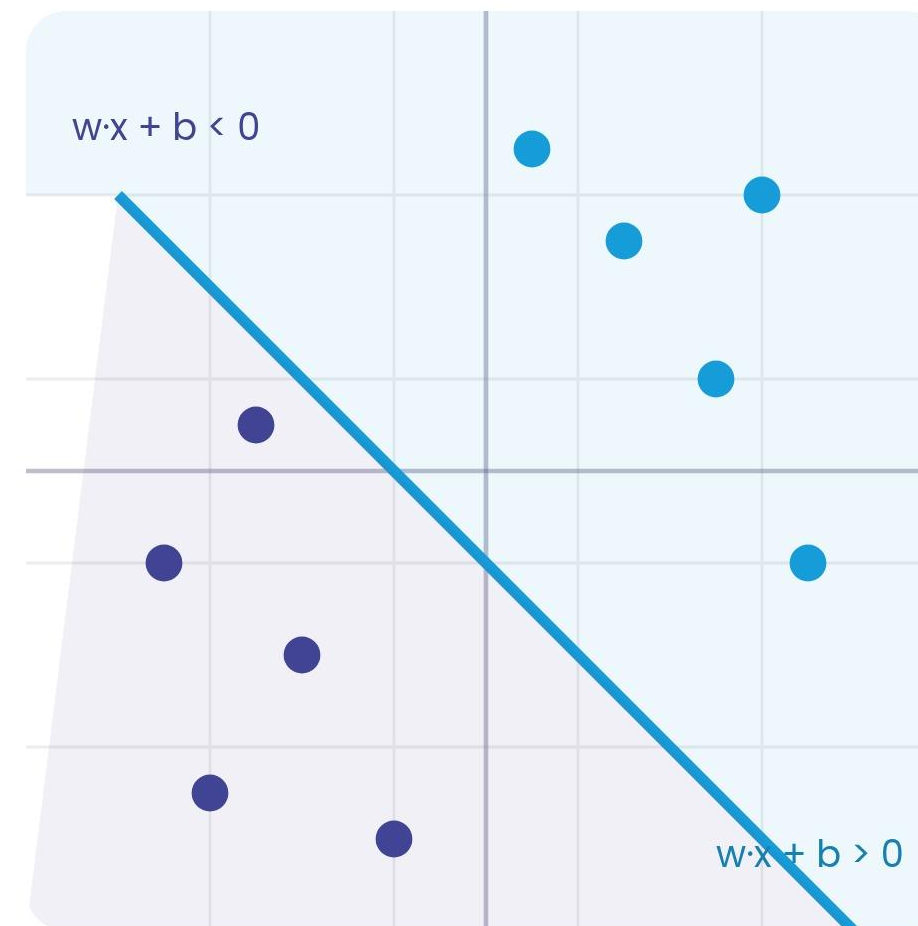
Chaque fonction transforme la somme pondérée en un signal utile : binaire, borné, ou passant.

Un neurone dessine un hyperplan

Dans l'espace des entrées, l'équation $\mathbf{w} \cdot \mathbf{x} + \mathbf{b} = 0$ définit une droite (2D), un plan (3D), un hyperplan (nD).

D'un côté, le neurone renvoie une valeur positive ; de l'autre, négative. La fonction d'activation traduit ensuite en décision.

Les poids déterminent l'orientation ; le biais, la position.



PARTIE 3 · 30 MIN

03

Le perceptron

Décision · Classification
binaire · Seuil · Frontière ·
Apprentissage · Mise à jour

Le perceptron, en une phrase

DÉFINITION

Un *perceptron* est un neurone artificiel muni d'une fonction seuil, entraîné à classer des exemples en deux catégories.

Frank Rosenblatt, 1957 : la première machine capable d'*apprendre* à trier des données à partir d'exemples annotés.

Deux classes, une décision

1

Spam ou pas ?

Fréquence de certains mots-clés, présence de liens, longueur du message.

2

Client à risque ?

Revenu, endettement, historique de paiements : accorder le prêt ou le refuser.

3

Tumeur bénigne ?

Rayon, texture, symétrie sur une image médicale : bénin ou malin.

La sortie du perceptron n'est pas un nombre continu : c'est un **oui** ou un **non**.

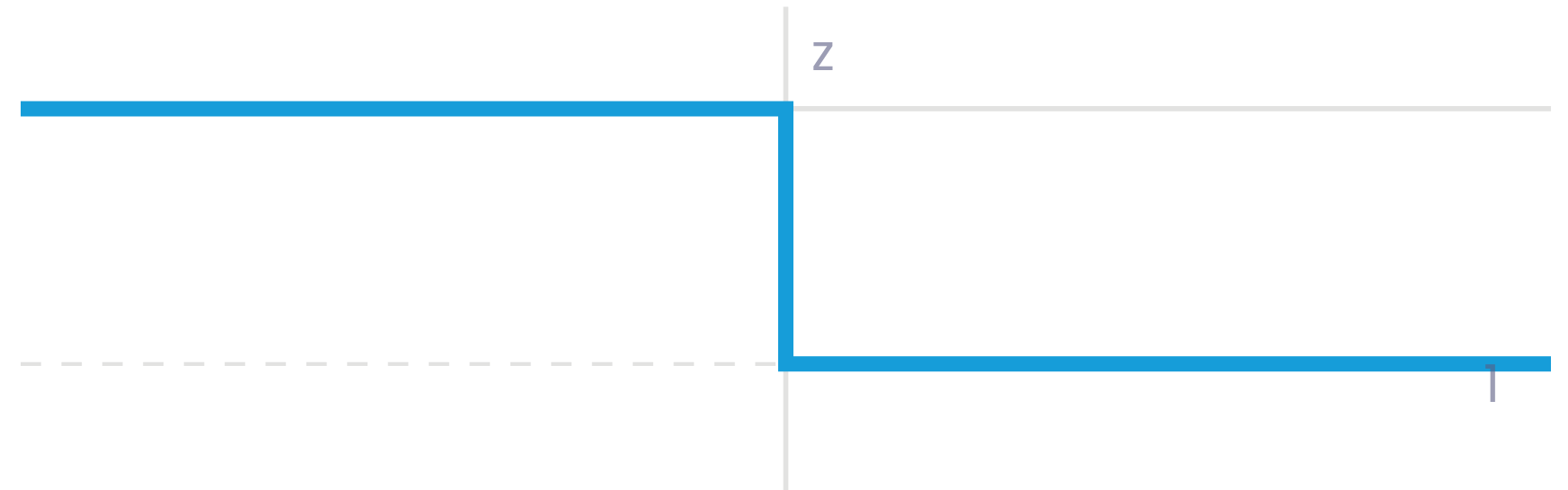
La fonction seuil : *Heaviside*

$$\hat{y} = f(z) =$$

$$1 \text{ si } z \geq 0$$

$$0 \text{ sinon}$$

$$\text{avec } z = \mathbf{w} \cdot \mathbf{x} + b$$

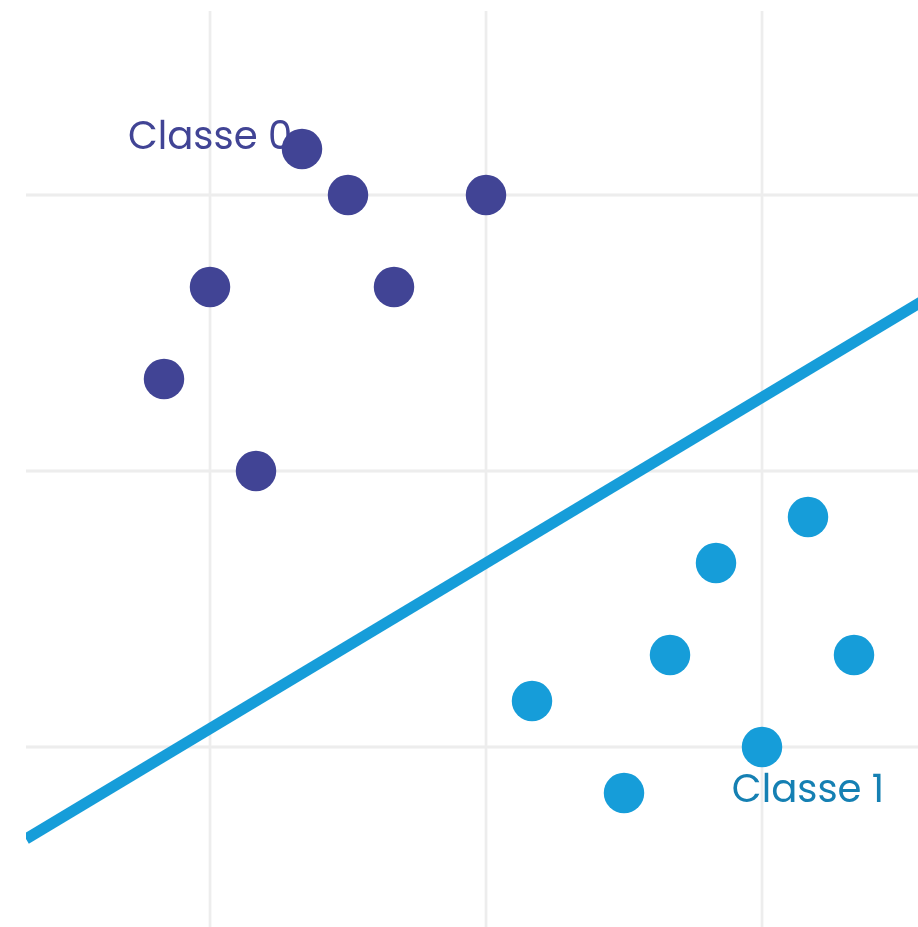


La frontière de décision

Le perceptron trace une **droite** (en 2D) qui sépare la classe 0 de la classe 1.

Cette droite est entièrement définie par les poids et le biais. Apprendre le perceptron, c'est trouver la **bonne** droite.

À retenir : l'orientation de la frontière vient des poids w ; sa position vient du biais b .



L'algorithme d'apprentissage

1

Initialiser les poids

Petits nombres aléatoires, ou zéros. La position de départ n'a pas d'importance capitale.

2

Prendre un exemple (x, y)

x est un vecteur de features, y la classe attendue (0 ou 1).

3

Prédire $\hat{y}$

Calculer $z = w \cdot x + b$, puis appliquer la fonction seuil pour obtenir $\hat{y}$.

4

Comparer et corriger

Si $\hat{y} \neq y$, mettre à jour les poids et le biais dans le sens qui aurait corrigé l'erreur.

5

Recommencer

Parcourir tous les exemples, plusieurs fois, jusqu'à convergence.

La règle de mise à jour

$$W_{\text{nouveau}} = W_{\text{ancien}} + \eta \cdot (y - \hat{y}) \cdot x$$

$$b_{\text{nouveau}} = b_{\text{ancien}} + \eta \cdot (y - \hat{y})$$

η : taux d'apprentissage (typiquement 0.01 à 0.1) · $(y - \hat{y})$: signe de l'erreur : +1, 0 ou -1

Bonne prédiction

$(y - \hat{y}) = 0$. Aucune correction : le perceptron laisse ses poids intacts.

Faux négatif

Attendu 1, prédit 0. On augmente les poids dans la direction de x pour la prochaine fois.

Faux positif

Attendu 0, prédit 1. On diminue les poids dans la direction de x .

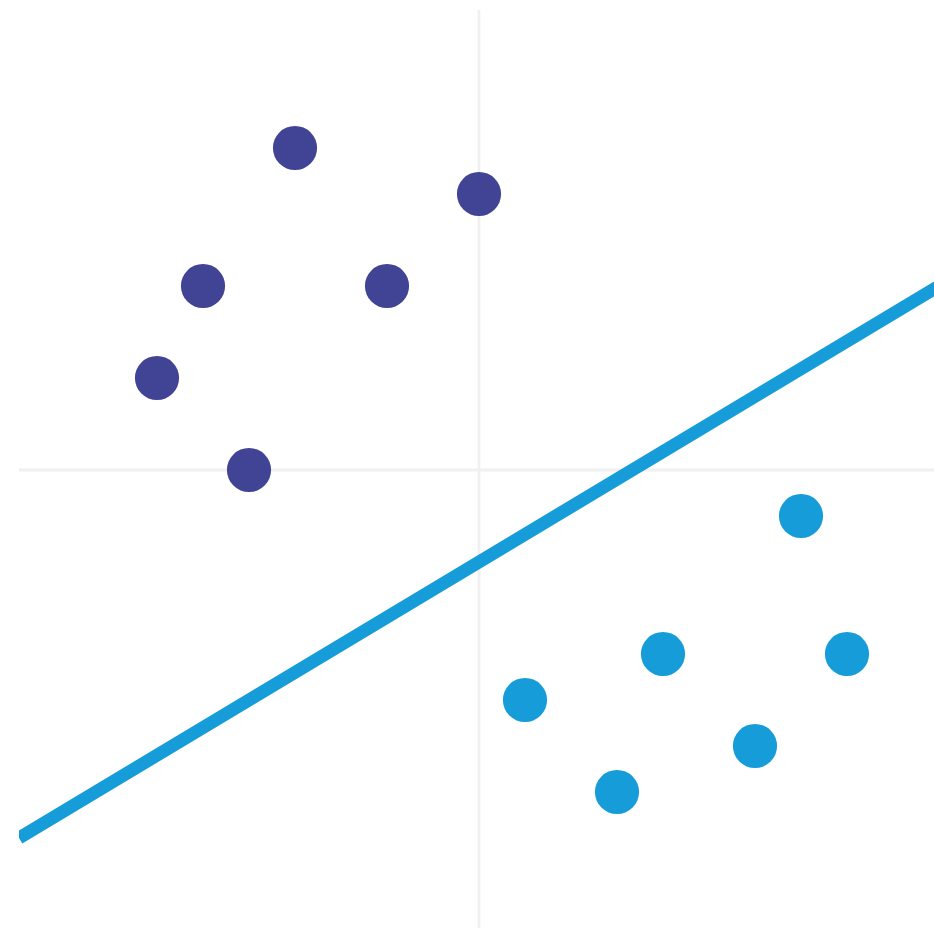
PARTIE 4 · 20 MIN

04

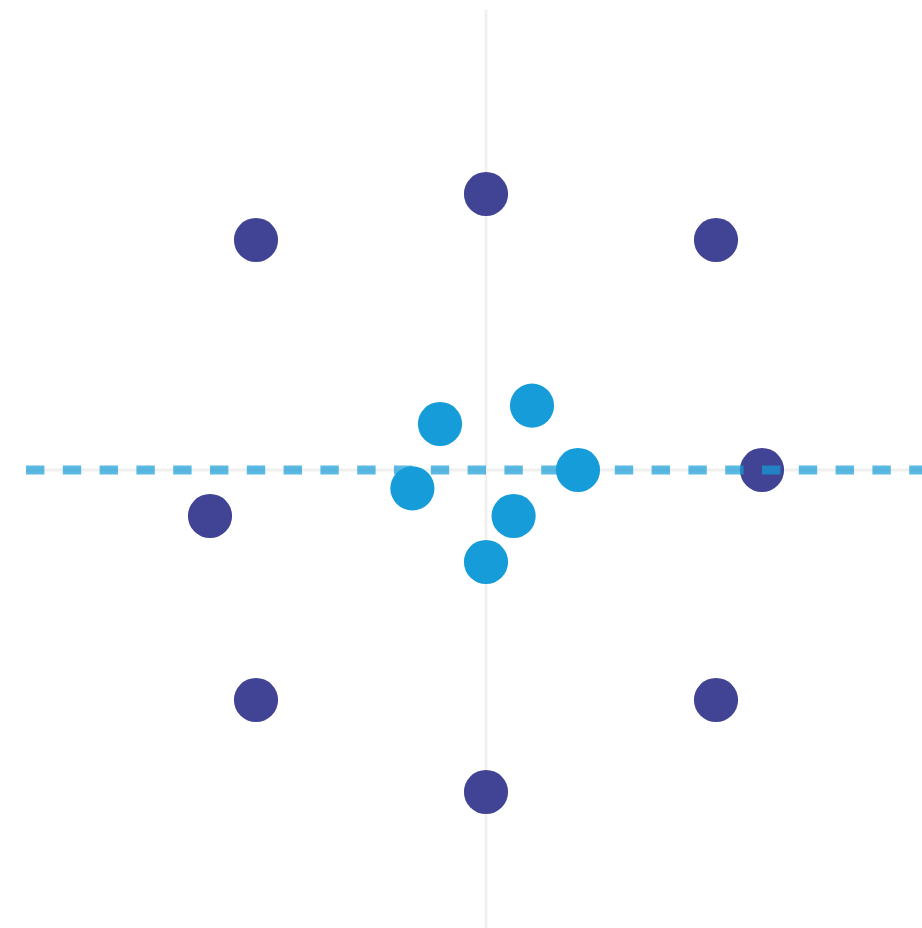
Les limites du perceptron

Séparabilité linéaire · Le cas XOR
· Besoin de plusieurs neurones

Une seule droite suffit-elle ?



Linéairement séparable ✓



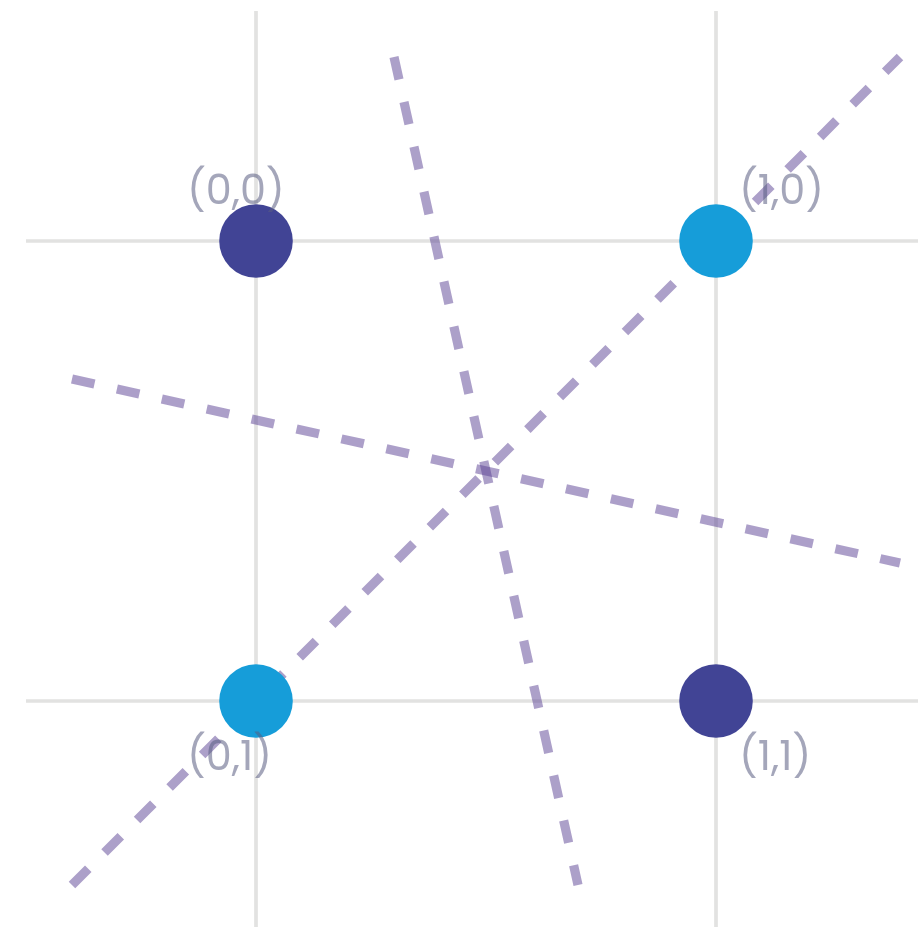
Non séparable linéairement ✗

XOR : quatre points qui font tomber le perceptron

XOR (« ou exclusif ») renvoie **1** quand exactement une des deux entrées vaut 1.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Aucune droite ne peut séparer les 1 des 0.



LA SORTIE

Une seule frontière ne peut pas tout séparer.
Il faut en combiner plusieurs.

C'est cette idée, **empiler des neurones**, qui donne naissance au perceptron multi-couches.

PARTIE 5 · 30 MIN

05

Du perceptron au MLP

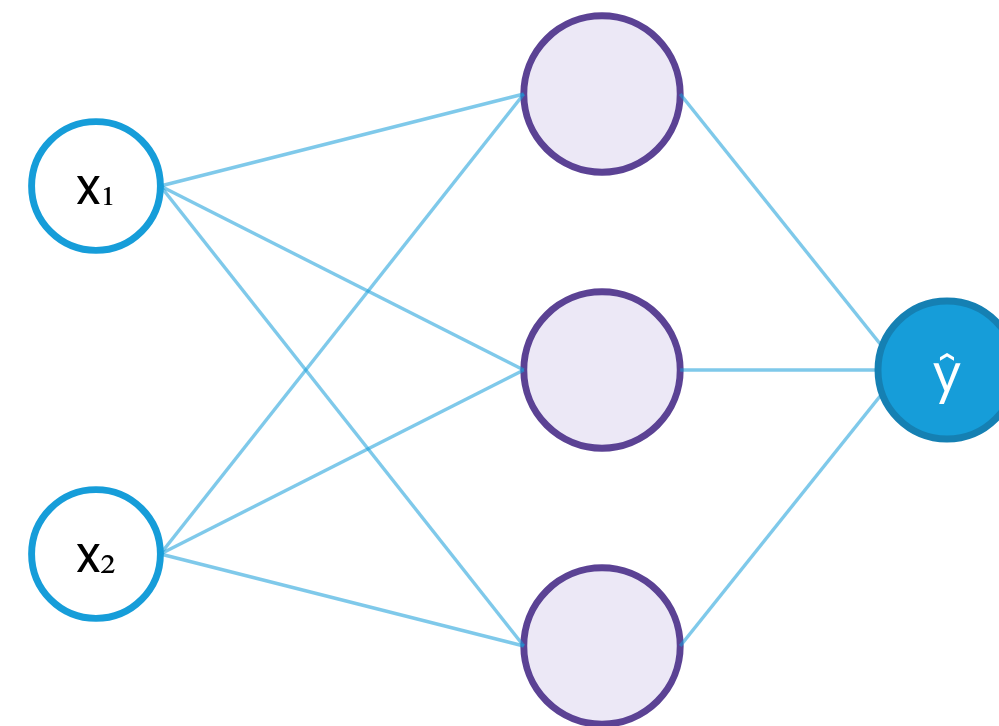
Plusieurs neurones · Couches
· Propagation · Non-linéarité
· Nombre de paramètres

Plusieurs neurones en parallèle

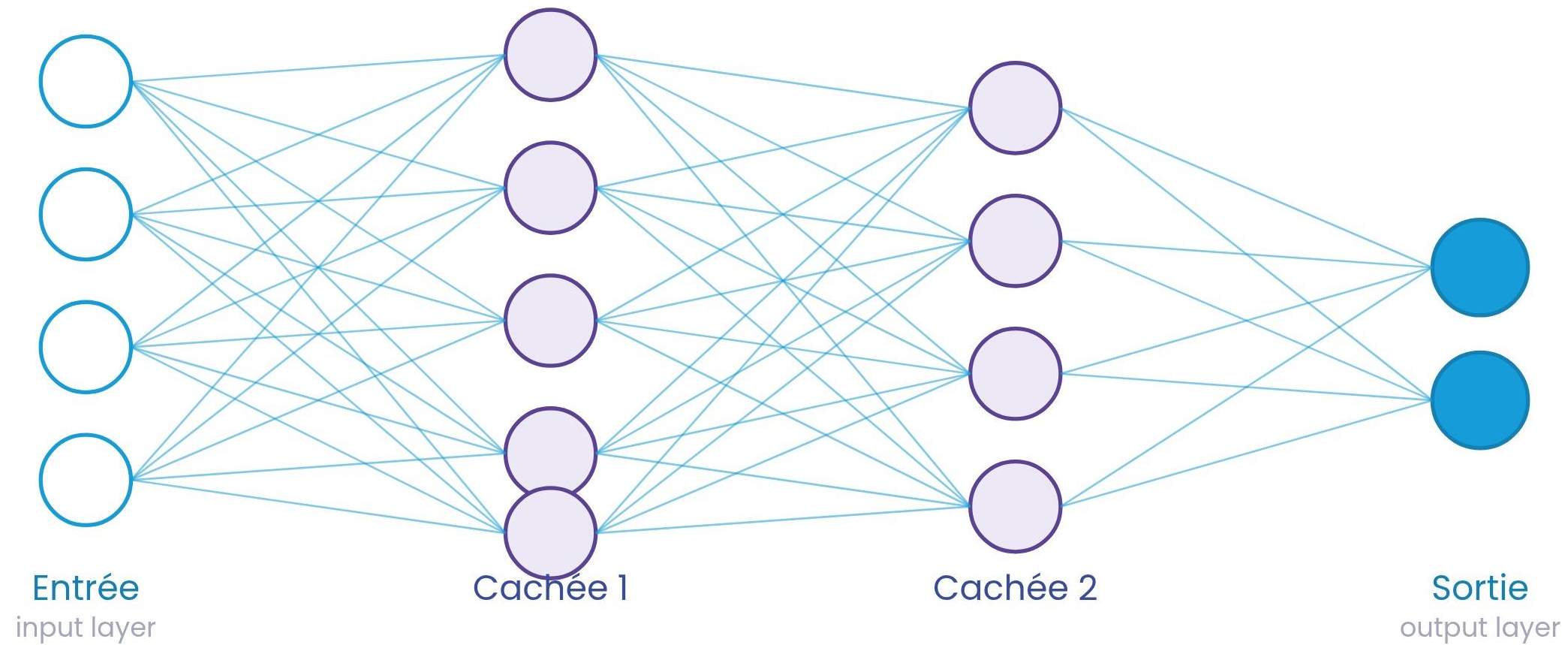
Un neurone = une frontière. **N neurones** côte à côte = N frontières.

Chaque neurone extrait un signal différent : « est-ce vertical ? », « est-ce à droite ? »... Ils constituent une nouvelle représentation des données.

Cette représentation devient l'entrée d'une couche suivante, qui recombine.



Couche d'entrée, cachée, de sortie



La propagation de l'information

1

Entrée

Le vecteur x arrive en tête du réseau.

2

Couche cachée $h_1 = f(W_1 \cdot x + b_1)$

Somme pondérée puis activation, pour chaque neurone de la couche.

3

Couche cachée $h_2 = f(W_2 \cdot h_1 + b_2)$

La sortie de la couche précédente devient l'entrée de la suivante.

4

Sortie $\hat{y} = g(W_3 \cdot h_2 + b_3)$

Une dernière activation, souvent différente (sigmoid, softmax, ou identité selon la tâche).

Pourquoi la non-linéarité est indispensable

FAIT MATHÉMATIQUE

Sans fonction d'activation non linéaire, empiler N couches revient à *une seule* couche.

Deux couches linéaires successives, soit $W_2 \cdot (W_1 \cdot x)$, se réduisent à une seule matrice $W_3 \cdot x$. La profondeur n'apporte rien.

Une non-linéarité entre les couches (ReLU, tanh...) casse cette réduction. Le réseau peut alors approcher **toute** fonction continue : théorème d'approximation universelle.

Combien de paramètres ?

COUCHE	ENTRÉES	NEURONES	POIDS	BIAIS	TOTAL
Hidden 1	784	128	100 352	128	100 480
Hidden 2	128	64	8 192	64	8 256
Output	64	10	640	10	650
Total réseau					109 386

Ex. classif. MNIST : image 28×28 = 784 pixels, 10 chiffres. À titre de comparaison, GPT-4 en compte plusieurs centaines de milliards.

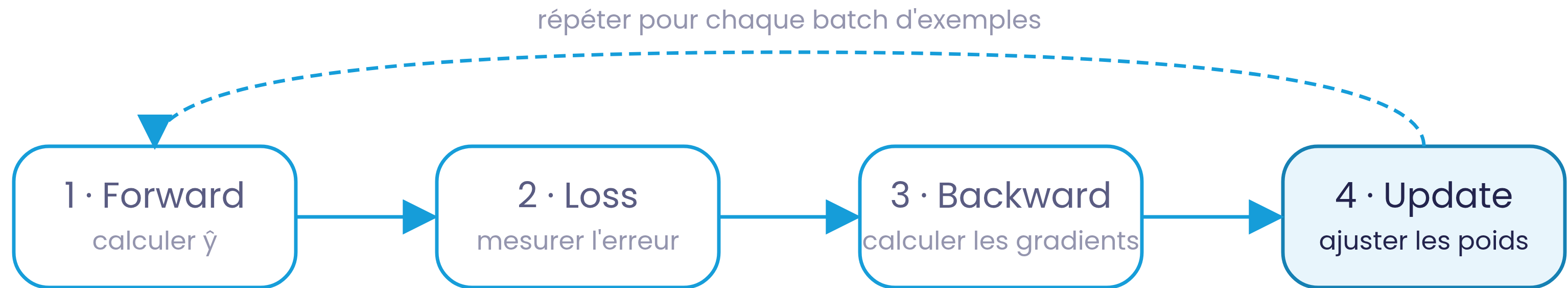
PARTIE 6 · 40 MIN

06

Comment un réseau de neurones apprend

Forward · Loss · Gradient ·
Descente · Backpropagation
· Optimizer · Mise à jour

La boucle d'apprentissage

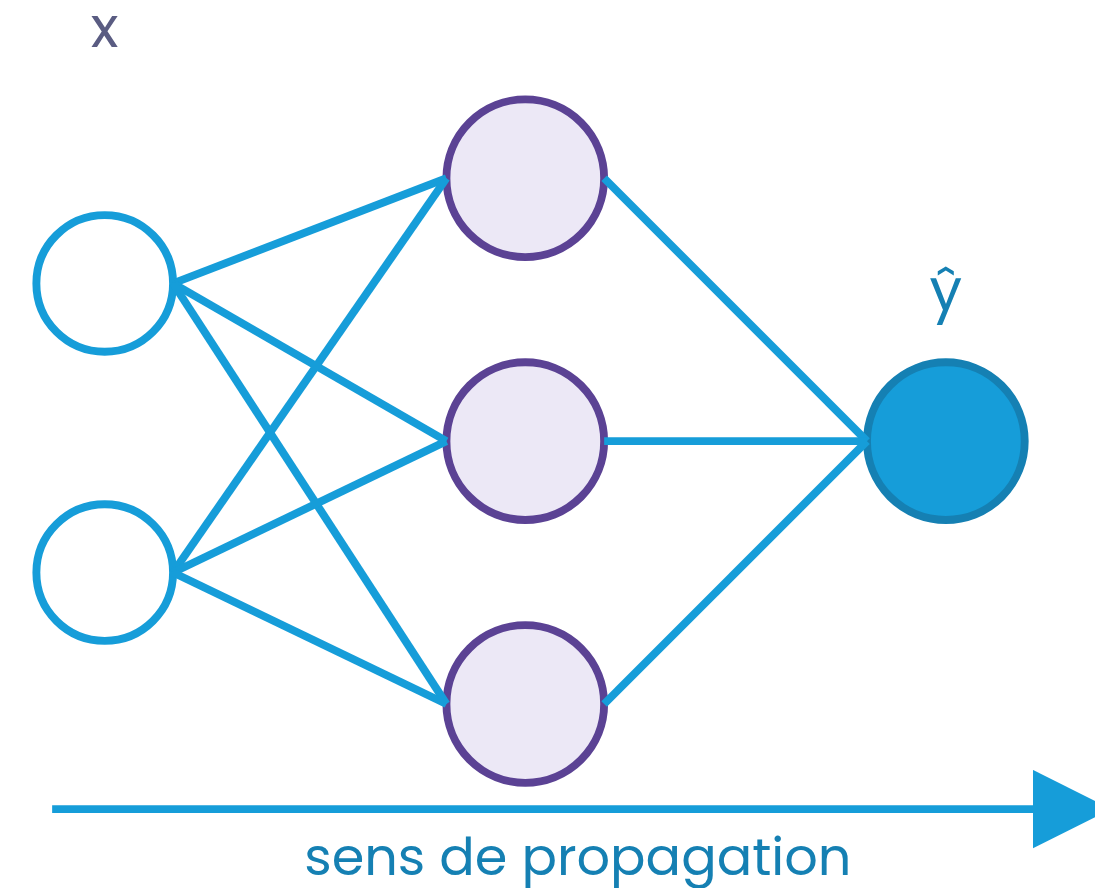


Forward pass : prédire

On fait passer un exemple $\mathbf{x}$ à travers le réseau, couche après couche, jusqu'à la sortie $\hat{\mathbf{y}}$.

C'est purement du calcul en avant : pas d'apprentissage à cette étape.

Tous les $\mathbf{z}$ et $\mathbf{h}$ intermédiaires sont mémorisés : ils serviront à la backpropagation.



Mesurer l'erreur : la fonction de coût

MSE

$$L = \frac{1}{2} \cdot (y - \hat{y})^2$$

Erreur quadratique moyenne : pour la régression. Pénalise plus fort les grosses erreurs.

Cross-entropy

$$L = -\sum y_i \cdot \log(\hat{y}_i)$$

Pour la classification. Explose quand la classe correcte reçoit une probabilité proche de 0.

Objectif

minimiser L

Un seul nombre à faire descendre : c'est tout ce que voit l'algorithme d'apprentissage.

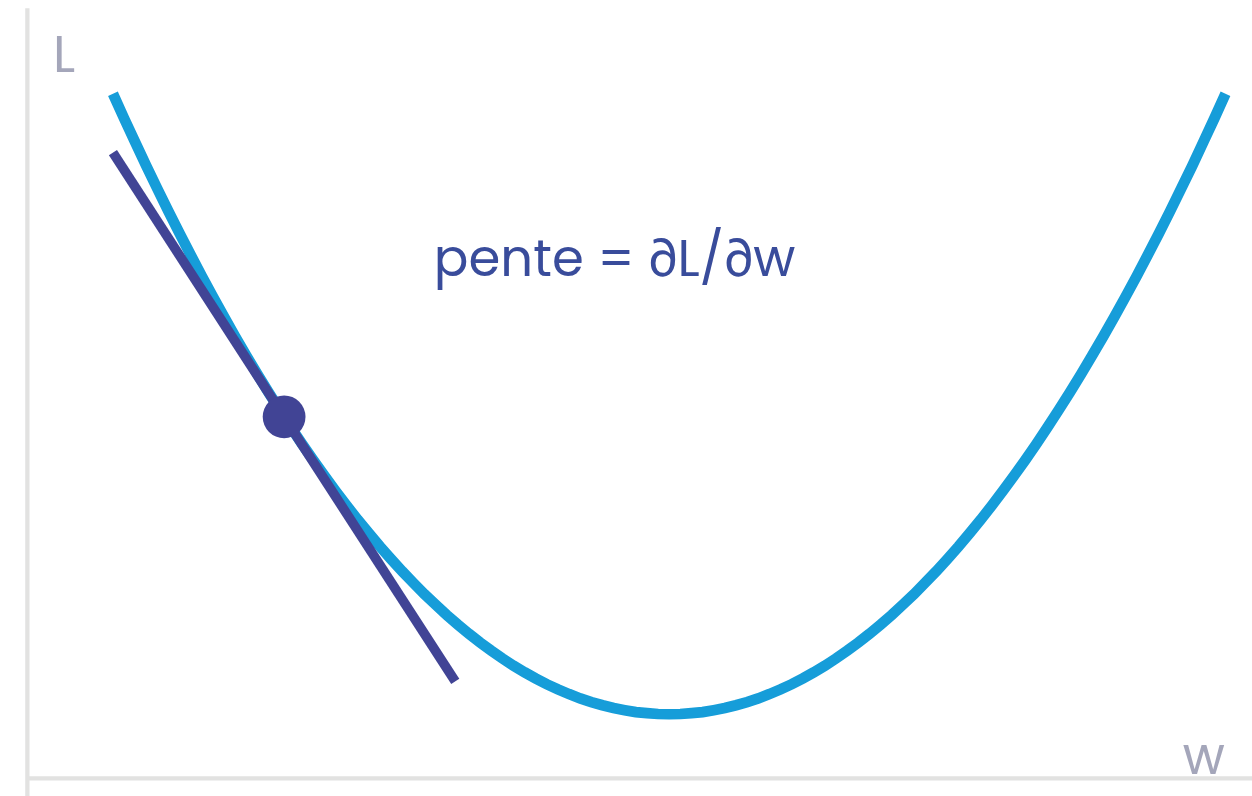
Le gradient : pente locale

Le **gradient** de la loss par rapport à un poids w , noté $\partial L / \partial w$, indique comment L varie si l'on bouge w .

Grand positif $\rightarrow$ augmenter w augmente L (donc il faut le diminuer). Grand négatif $\rightarrow$ l'inverse.

$$\nabla L = (\partial L / \partial w_1 , \partial L / \partial w_2 , \dots)$$

un vecteur : une composante par paramètre.

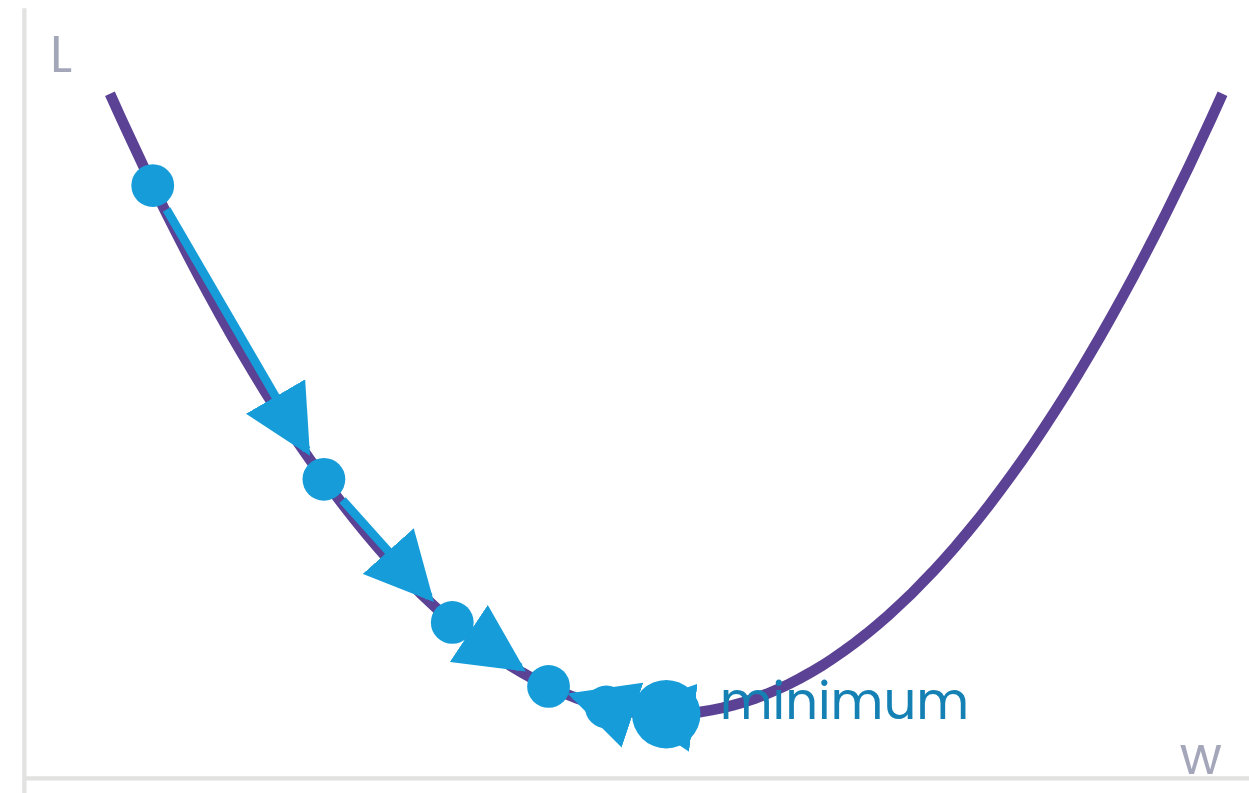


La descente de gradient

$$w \leftarrow w - \eta \cdot \partial L / \partial w$$

On soustrait le gradient car on veut *descendre*, aller vers un L plus bas.

η : le *taux d'apprentissage*. Trop petit : convergence lente. Trop grand : on saute par-dessus le minimum.

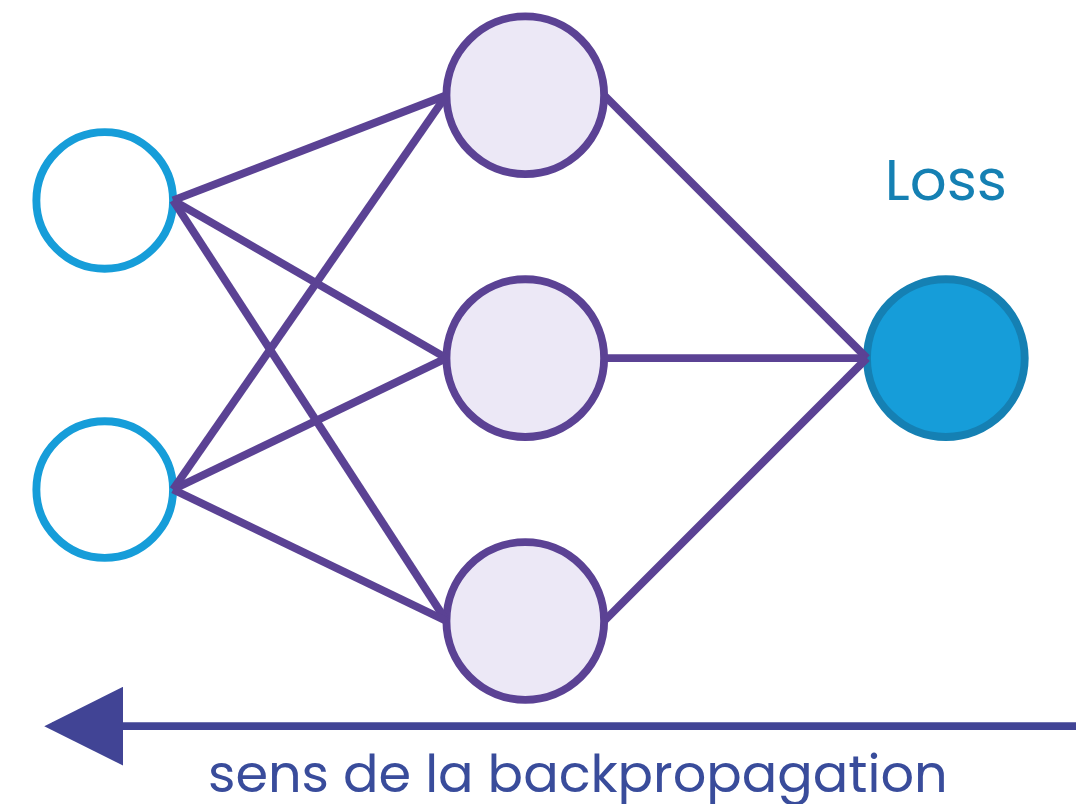


Backpropagation : l'erreur remonte

Pour ajuster les poids, il faut savoir **combien** chaque poids contribue à l'erreur.

La backpropagation applique la règle de la chaîne pour distribuer l'erreur, de la sortie vers l'entrée. En un seul balayage arrière, on obtient *tous* les gradients.

Redécouverte en 1986 par Rumelhart, Hinton et Williams : l'algorithme qui a débloqué le deep learning.



L'optimizer : la stratégie de descente

SGD

$$w \leftarrow w - \eta \cdot \nabla L$$

Stochastic Gradient Descent : le pas est un multiple constant du gradient. Simple, mais sensible au taux d'apprentissage.

Momentum

Ajoute une **inertie** : le pas courant est influencé par les précédents. Traverse plus vite les zones plates.

Adam

Choix par défaut. Ajuste automatiquement un pas par paramètre, selon la moyenne et la variance des gradients passés. Robuste.

Mise à jour : la boucle se referme

pour chaque paramètre θ du réseau :

$$\theta \leftarrow \theta - \eta \cdot \partial L / \partial \theta$$

tous les poids et biais, dans toutes les couches, sont mis à jour *simultanément*.

- Un passage complet sur les données = **une époque** : le réseau en verra typiquement des dizaines.
- La loss doit descendre au fil des époques. Sinon : ajuster η , revoir les données, changer d'architecture.
- On surveille aussi la loss sur un jeu de *validation* : pour détecter le sur-apprentissage.

PARTIE 7 · 10 MIN

07

Synthèse

Du neurone au MLP · La
boucle d'apprentissage
· À retenir · Transition TP

Du neurone au MLP

1

Le neurone artificiel

Somme pondérée des entrées + biais, passée dans une fonction d'activation.

2

Le perceptron

Un neurone avec fonction seuil, entraîné à classer deux catégories linéairement séparables.

3

La couche

Plusieurs neurones en parallèle, chacun apprenant une caractéristique différente.

4

Le MLP

Plusieurs couches empilées, avec une non-linéarité entre chaque, capable d'approcher toute fonction.

Les quatre idées à emporter

1

Apprendre = ajuster des paramètres

Tout se joue dans les poids et les biais. Rien d'autre ne change pendant l'entraînement.

2

La loss guide tout

Un seul nombre, l'erreur, dirige toute la mise à jour du réseau, quelle que soit sa taille.

3

La non-linéarité fait la puissance

Sans activation non linéaire, la profondeur n'apporte rien. C'est elle qui fait le deep learning.

4

Le gradient dit où aller

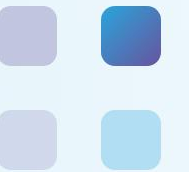
La backpropagation calcule le gradient de la loss par rapport à chaque paramètre : le reste est arithmétique.

Cet après-midi : travaux pratiques

PROGRAMME DU TP

Coder un perceptron « à la main » ·
Entraîner un MLP sur MNIST avec PyTorch ·
Explorer l'effet du taux d'apprentissage.

Objectif : lier chaque ligne de code à un concept vu ce matin.



FIN DE LA MATINÉE

Merci

Questions ouvertes avant la pause,
puis TP à 14 h en salle machines.